Secure Authentication Lab

A small local authentication service in Node.js and Express with safe password storage, server-side validation, secure cookies, session expiry, rate limiting and generic errors. Every control was tested; the real output is in section 4.

1. Requirements checklist

Lab step	What I built	Verified by
Password hashing	bcrypt, cost 12, automatic per-password salt. Plain passwords are never stored or logged.	Register works; hash only in memory store
Server-side validation	Email format and length, password 12–128 chars, common-password check, strict type checks (rejects objects and arrays).	Short password and `{"$ne":1}` payload both rejected
Secure cookies	`HttpOnly`, `SameSite=Strict`, `Secure` in production, custom name `sid`.	Set-Cookie header in section 4
Logout	Destroys the server-side session and clears the cookie.	`/me` returns 401 after logout
Session expiry	15-minute idle timeout (rolling) plus 8-hour absolute limit. Session ID regenerated on login.	Cookie Expires about 15 min ahead
Rate limits	10 failed logins per IP per 15 min, 10 registrations per IP per hour, plus per-account lockout after 5 failures.	6th wrong attempt returns 429
Generic errors	One message for wrong email or wrong password, constant-time bcrypt check for unknown users, no stack traces.	Unknown user gets the same message

2. Run it

2. Run it

```
mkdir secure-auth-lab && cd secure-auth-lab
# save package.json and server.js from sections 3 and 5, then:
npm install
npm start          # http://127.0.0.1:3000
```

For production, run behind HTTPS with `NODE_ENV=production` and a fixed `SESSION_SECRET`. Locally the cookie is not `Secure` so it works over plain HTTP.

---

3. server.js

```
'use strict';
const express = require('express');
const session = require('express-session');
const rateLimit = require('express-rate-limit');
const helmet = require('helmet');
const bcrypt = require('bcryptjs');
const crypto = require('crypto');

const PORT = process.env.PORT || 3000;
const IS_PROD = process.env.NODE_ENV === 'production';
const BCRYPT_COST = 12;
const IDLE_TIMEOUT_MS = 15 * 60 * 1000;        // session dies after 15 min of inactivity
const ABSOLUTE_TIMEOUT_MS = 8 * 60 * 60 * 1000; // ...and never lives longer than 8 hours
const LOCK_AFTER = 5;                           // failed logins before a temporary lock
const LOCK_MS = 15 * 60 * 1000;

const app = express();
app.disable('x-powered-by');
app.use(helmet());
app.use(express.json({ limit: '10kb' }));

// ---- In-memory "database" (use a real DB in production) ----
const users = new Map(); // email -> { id, email, hash }
const failures = new Map(); // email -> { count, until }
const DUMMY_HASH = bcrypt.hashSync(crypto.randomBytes(16).toString('hex'), BCRYPT_COST);
const COMMON = new Set(['password1234', '123456789012', 'qwertyuiop12', 'letmein12345']);

// ---- Sessions: secure cookie settings + expiry ----
app.use(session({
  name: 'sid',
```

```
  secret: process.env.SESSION_SECRET || crypto.randomBytes(32).toString('hex'),
  resave: false,
  saveUninitialized: false,
  rolling: true, // refresh the idle timer on every request
  cookie: {
    httpOnly: true,                // JavaScript cannot read the cookie (limits XSS dama
    secure: IS_PROD,               // HTTPS only in production
    sameSite: 'strict',            // not sent on cross-site requests (CSRF defence)
    maxAge: IDLE_TIMEOUT_MS,
  },
}));

// ---- Rate limits ----
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, limit: 10, standardHeaders: true, legacyHeaders: false,
  skipSuccessfulRequests: true,
  message: { error: 'Too many attempts. Please try again later.' },
});
const registerLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, limit: 10, standardHeaders: true, legacyHeaders: false,
  message: { error: 'Too many attempts. Please try again later.' },
});

// ---- Server-side validation ----
function validateNewAccount(body) {
  const { email, password } = body || {};
  const errors = [];
  if (typeof email !== 'string' || email.length > 254 || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.te
    errors.push('A valid email is required.');
  }
  if (typeof password !== 'string' || password.length < 12 || password.length > 128) {
    errors.push('Password must be 12 to 128 characters.');
  } else if (COMMON.has(password.toLowerCase())) {
    errors.push('Password is too common.');
  }
  return errors;
}

function isLocked(email) {
  const f = failures.get(email);
  return Boolean(f && f.until > Date.now());
}
function recordFailure(email) { // tracked for unknown emails too, so lockout leaks nothi
  const f = failures.get(email) || { count: 0, until: 0 };
  f.count += 1;
  if (f.count >= LOCK_AFTER) { f.count = 0; f.until = Date.now() + LOCK_MS; }
  failures.set(email, f);
```

```
}

function requireAuth(req, res, next) {
  const s = req.session;
  if (!s.userId || Date.now() - s.createdAt > ABSOLUTE_TIMEOUT_MS) {
    return s.destroy(() => res.status(401).json({ error: 'Not authenticated.' }));
  }
  next();
}

// ---- Routes ----
app.post('/register', registerLimiter, async (req, res, next) => {
  try {
    const errors = validateNewAccount(req.body);
    if (errors.length) return res.status(400).json({ errors });
    const email = req.body.email.trim().toLowerCase();
    if (users.has(email)) return res.status(400).json({ errors: ['Could not create accoun
    const hash = await bcrypt.hash(req.body.password, BCRYPT_COST); // salted automatical
    users.set(email, { id: crypto.randomUUID(), email, hash });
    res.status(201).json({ message: 'Account created.' });
  } catch (err) { next(err); }
});

app.post('/login', loginLimiter, async (req, res, next) => {
  try {
    const { email, password } = req.body || {};
    if (typeof email !== 'string' || typeof password !== 'string' || email.length > 254 |
      return res.status(400).json({ error: 'Invalid request.' });
    }
    const key = email.trim().toLowerCase();
    if (isLocked(key)) return res.status(429).json({ error: 'Too many attempts. Please tr

    const user = users.get(key);
    // Always run bcrypt, even for unknown users, so timing does not reveal which emails
    const ok = await bcrypt.compare(password, user ? user.hash : DUMMY_HASH);
    if (!user || !ok) {
      recordFailure(key);
      return res.status(401).json({ error: 'Invalid email or password.' }); // same messa
    }
    failures.delete(key);
    req.session.regenerate((err) => { // new session ID on login prevents session fixatio
      if (err) return next(err);
      req.session.userId = user.id;
      req.session.createdAt = Date.now();
      res.json({ message: 'Logged in.' });
    });
  } catch (err) { next(err); }
```

```
  } catch (err) { next(err); }
});

app.get('/me', requireAuth, (req, res) => res.json({ userId: req.session.userId }));

app.post('/logout', (req, res, next) => {
  req.session.destroy((err) => {
    if (err) return next(err);
    res.clearCookie('sid');
    res.status(204).end();
  });
});

// ---- Generic error handler: no stack traces to the client ----
app.use((err, req, res, next) => { // eslint-disable-line no-unused-vars
  console.error(err.message);
  const bad = err.status === 400;
  res.status(bad ? 400 : 500).json({ error: bad ? 'Invalid request.' : 'Something went wr
});

app.listen(PORT, '127.0.0.1', () => console.log(`Auth lab running on http://127.0.0.1:${P
```

4. Test results (real output)

```
$ curl -X POST /register -d '{"email":"a@b.com","password":"short"}'
{"errors":["Password must be 12 to 128 characters."]}

$ curl -X POST /login -d '{"email":{"$ne":1},"password":"x"}'
{"error":"Invalid request."}

$ curl -X POST /register -d '{"email":"a@b.com","password":"CorrectHorse-Battery9"}'
{"message":"Account created."}

$ curl -i /me                       # before login
HTTP/1.1 401 Unauthorized

$ curl -i -c jar /login (correct credentials)
HTTP/1.1 200 OK
Set-Cookie: sid=s%3A...; Path=/; Expires=(about 15 min ahead); HttpOnly; SameSite=Strict

$ curl -b jar /me
{"userId":"53b84af9-a7b7-4056-b81b-96ed9c5c079f"}
```

```
$ curl -i -b jar -X POST /logout
HTTP/1.1 204 No Content

$ curl -b jar /me                    # after logout
{"error":"Not authenticated."}

6 wrong passwords in a row  ->  401 401 401 401 401 429

$ curl -X POST /login (email that does not exist)
{"error":"Invalid email or password."}
```

5. package.json

```
{
  "name": "secure-auth-lab",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "express": "^4.19.2",
    "express-rate-limit": "^7.4.0",
    "express-session": "^1.18.0",
    "helmet": "^7.1.0"
  }
}
```

6. Short explanation

Passwords. I hash with bcrypt because it is slow on purpose and adds a unique salt, so a stolen database cannot be cracked quickly or with shared lookup tables. Validation runs on the server because the browser can be bypassed, and type checks stop object-injection payloads.

Sessions. The session lives on the server and the browser only holds a random ID in a

cookie. `HttpOnly` keeps scripts away from it, `SameSite=Strict` blocks cross-site requests, and `Secure` forces HTTPS in production. A new ID is issued at login to stop session fixation, and sessions expire on idle and absolute timers. Logout destroys the session on the server, so a copied cookie stops working.

Abuse protection. Rate limits slow down guessing from one IP, and the per-account lockout covers attackers who rotate IPs. Every failure returns the same message and runs the same bcrypt work, so attackers cannot learn which emails are registered.

Known limits. Users and sessions are in memory, so a restart clears them. A real deployment needs a database, a persistent session store such as Redis, MFA, email verification and password reset, and audit logging.